📄 PlacementConnect - Functional Requirements Document (FRD)

1. System Context and Architecture

- **Frontend:** Vue 3 SPA, Vite build, Vue Router navigation, Pinia state stores and Axios API services.
- **Backend:** Flask application exposing Flask-RESTX namespaces for authentication, admin, company and student operations.
- **Persistence:** SQLAlchemy models and relational constraints.
- **Caching/ Background processing:** Redis dashboard/notification caching; Celery tasks for background notification, reminder, report and export operations; mail and generated-file storage.

2. Permissions

Actor	Allowed functional areas
Guest	Public landing/role entry, Student/Company registration and login
Admin	Admin dashboard, profile, inbox, company/student management, drive approvals, application inspection
Company	Company dashboard, own profile, own drives, applicants for own drives, application processing, inbox
Student	Student dashboard, own profile, approved company/drive discovery, own applications, inbox

Authorisation requirement: Frontend route guards are a navigation aid. Backend decorators and service-level ownership filters must remain the authoritative security boundary.

3. Functional Modules

3.1 Registration and Authentication

Inputs

- Registration: username, email, password, role.
- Login: username or email, password, role.

Processing

1. Validate username, email and password.
2. Permit public registration only for Company and Student roles.
3. Reject duplicate username or email.
4. Hash password before persistence.
5. Create the role-specific profile record.
6. On login, locate account by username or email, verify password, validate role, and check account status.
7. For Company role, require company approval status to be Approved.
8. Return authenticated user/session information.
9. Logout invalidates the session; current-user endpoint returns current authenticated identity.

Validation

- Username: 2–50 characters; lowercase letters, numbers, underscore and hyphen pattern.
- Email: 8–100 characters and a restrictive lowercase pattern.
- Password: 6–255 characters and restricted character set.
- Duplicate username/email rejected.

Implementation behaviour: Company and Student registration creates profile records with placeholder/blank profile attributes. Registration is therefore not equivalent to profile completion. Admin registration is not allowed through the public registration service.

3.2 Admin Dashboard and Management

Admin dashboard provides aggregate placement information. Admin list/detail capabilities include:

- List companies.
- List students.
- List drives for a company.
- List applications for a company-drive pair.
- List applications for a student.
- Retrieve application detail.
- Approve/reject/reapprove company.
- Disable/enable/blacklist/whitelist/delete company.
- Disable/enable/blacklist/whitelist/delete student.

- Approve/reject/reapprove drive.
- View/update admin profile.
- Start admin profile export.
- View inbox, notification detail, attachments and notification status.

Business logic

- Company approval actions update the Company approval_status.
- Company and student account actions update the related User.status.
- Drive approval actions update Drive.approval_status.
- Delete operations remove the associated user/profile and may cascade to dependent records according to ORM relationships and database configuration.
- Dashboard caches are cleared after relevant changes.

3.3 Company Profile and Drive Management

Company functions:

- View/update own profile.
- Delete own profile.
- Start profile export.
- View dashboard.
- List own drives.
- Create, update and delete own drives.
- List applications for a drive.
- Retrieve application detail.
- Update an application and its interview/placement records.
- Access own inbox and notification attachments.

Drive fields

- Job title, job description, location, job type, job mode, salary package.
- Eligibility criteria, optional minimum CGPA.
- Application start and deadline.
- Company relationship and approval status.
- Job type: Permanent, Internship, Contract.
- Job mode: Onsite, Remote, Hybrid.
- Drive approval: Pending, Approved, Rejected.

Drive validation

- Salary must be a non-negative number.
- Minimum CGPA, when supplied, must be between 0 and 10.

- Application start and deadline must be future date-times at creation/update.
- Start must be before deadline.
- Date-times are interpreted using IST.

3.4 Student Profile and Opportunity Discovery

Student profile data includes:

- Name and contact number.
- Degree, department, graduation year and CGPA.
- Skills, certifications and achievements.
- LinkedIn, GitHub and portfolio links.
- Resume reference.
- Related education, experience and project records.

Student functions:

- View/update profile.
- Delete profile.
- Start profile export.
- View dashboard.
- Browse approved active companies.
- Browse approved drives for a company.
- View drive details.
- Apply and withdraw.
- View inbox and attachments.

Discovery filtering

- Company must be Approved and its User account Active.
- Drive must be Approved.
- Drive detail must also be associated with an approved company and active company account.
- Drive phase is derived from current IST time:
  - Upcoming: now is before application_start.
  - Ongoing: now is at or after application_start and at or before application_deadline.
  - Closed: now is after application_deadline.

Eligibility

- Eligible if minimum_cgpa is null or student.cgpa >= minimum_cgpa.
- Ineligible students may view drive details but cannot apply.

3.5 Application Lifecycle

Submission preconditions

- Drive exists and is visible through approved company/drive state.
- Current time is not before application_start.
- Current time is not after application_deadline.
- Student meets minimum CGPA if configured.
- No existing application exists for the same student and drive.

Submission result

- Create Application with initial status Applied.
- Enforce database uniqueness on (student_id, drive_id).
- Clear relevant dashboard caches.

Withdrawal preconditions

- Application exists for the student and drive.
- Drive/company remains approved and company account active under the query conditions.
- Current time is within the application window.
- Application status is Applied.

Withdrawal result

- Delete the application record.
- Clear relevant dashboard caches.

3.6 Recruiter Application Processing

Application status

- Initial: Applied.
- Applied → Rejected.
- Applied → Shortlisted, only when interview mode, start date-time and end date-time are provided.
- Shortlisted and Rejected are treated as terminal application states.

Interview

- Created when application is shortlisted.
- Initial status: Scheduled.
- Mode: Virtual, On-Campus or Off-Campus.
- Scheduled → Not Selected.
- Scheduled → Selected, only when joining date-time is provided.
- Selected and Not Selected are terminal interview states.

Placement

- Created when interview status changes to Selected.
- Initial status: Offered.
- Offered → Accepted or Declined.
- Joining date-time can be updated while status remains Offered.
- Accepted and Declined are terminal placement states.

Date validation

- Supplied interview and joining date-times must be future date-times.
- When both interview start and end are supplied, start must precede end.

3.7 Inbox, Notifications, Reports and Exports

Each role exposes:

- Notification summary list.
- Notification detail retrieval.
- Attachment retrieval.
- Notification status endpoint.

Profile export operations are exposed for Admin, Company and Student. Background task modules cover notification, reminder, report and export processing. Generated storage paths are organised by category (exports, reminders, reports; archived notifications).

4. API

The following paths are relative to their role namespace.

Authentication

- `POST /register`
- `POST /login`
- `POST /logout`
- `GET /current_user`

Admin

- `GET /dashboard`
- `GET /companies`
- `GET /students`
- `GET /companies/{company_id}/drives`
- `GET /companies/{company_id}/drives/{drive_id}/applications`
- `GET /students/{student_id}/applications`
- `GET /applications/{application_id}`

- `POST /companies/{company_id}/approve`
- `POST /companies/{company_id}/reject`
- `POST /companies/{company_id}/reapprove`
- `POST /companies/{company_id}/disable`
- `POST /companies/{company_id}/enable`
- `POST /companies/{company_id}/blacklist`
- `POST /companies/{company_id}/whitelist`
- `DELETE /companies/{company_id}/delete`
- `POST /students/{student_id}/disable`
- `POST /students/{student_id}/enable`
- `POST /students/{student_id}/blacklist`
- `POST /students/{student_id}/whitelist`
- `DELETE /students/{student_id}/delete`
- `POST /drives/{drive_id}/approve`
- `POST /drives/{drive_id}/reject`
- `POST /drives/{drive_id}/reapprove`
- `GET /profile`, `PUT /profile`, `POST /profile/export`
- `GET /inbox`
- `GET /inbox/notification/{notification_filename}`
- `GET /inbox/notification/{notification_filename}/attachment/{attachment_filename}`
- `GET /inbox/status`

Company

- `GET /dashboard`
- `GET /drives`
- `GET /drives/{drive_id}/applications`
- `GET /drives/{drive_id}/applications/{application_id}`
- `POST /drives/create`
- `PUT /drives/{drive_id}/update`
- `DELETE /drives/{drive_id}/delete`
- `PUT /applications/{application_id}/update`
- `GET /profile`, `PUT /profile`, `DELETE /profile`, `POST /profile/export`
- `GET /inbox`
- `GET /inbox/notification/{notification_filename}`

- `GET /inbox/notification/{notification_filename}/attachment/{attachment_filename}`
- `GET /inbox/status`

Student

- `GET /dashboard`
- `GET /companies`
- `GET /companies/{company_id}/drives`
- `GET /companies/{company_id}/drives/{drive_id}`
- `POST /drives/{drive_id}/apply`
- `DELETE /drives/{drive_id}/withdraw`
- `GET /profile`, `PUT /profile`, `DELETE /profile`, `POST /profile/export`
- `GET /inbox`
- `GET /inbox/notification/{notification_filename}`
- `GET /inbox/notification/{notification_filename}/attachment/{attachment_filename}`
- `GET /inbox/status`

5. Data Model and Relationships

Entity	Purpose	Key relationships
User	Authentication, role and account status	One-to-one Company or Student profile
Company	Recruiter organisation details and approval	Has many Drives
Student	Student placement profile	Has many Applications, Education, Experience and Projects
Education	Academic history	Belongs to Student
Experience	Work/internship history	Belongs to Student
Project	Student project portfolio	Belongs to Student
Drive	Job opportunity and application window	Belongs to Company; has many Applications

Application	Student's application to a Drive	Belongs to Student and Drive; optional one Interview
Interview	Interview schedule and result	One-to-one Application; optional one Placement
Placement	Offer and joining information	One-to-one Interview

6. Security and Data Integrity Requirements

- Passwords must never be stored or returned in plaintext.
- Role checks must be enforced server-side.
- Admin-only operations must be protected.
- Unique username/email and unique student-drive application constraints must be enforced.
- Notification and attachment access must be constrained to the owning user/role.
- Deletion and cascade behaviour must be reviewed against retention and audit needs.

7. Traceability to Code

Concern	Implementation location
Database entities	`backend/app/database/models.py`
Authentication logic	`backend/app/services/auth.py`, `backend/app/resources/auth.py`, `backend/app/schemas/auth.py`
Admin operations	`backend/app/services/admin.py`, `backend/app/resources/admin.py`, `backend/app/schemas/admin.py`

Company operations	`backend/app/services/company.py`, `backend/app/resources/company.py`, `backend/app/schemas/company.py`
Student operations	`backend/app/services/student.py`, `backend/app/resources/student.py`, `backend/app/schemas/student.py`
Validation	`backend/app/utils/validators.py`
API registration	`backend/app/main.py`
Frontend routes and role guards	`frontend/src/router/index.js`
Frontend API clients	`frontend/src/services/`
Frontend views	`frontend/src/views/`
Frontend Vue Components	`frontend/src/components/`
Frontend Pinia Store/ Flash Message	`frontend/src/stores/`
Background tasks	`backend/app/integrations/processing/task/`
Cache integration	`backend/app/integrations/processing/cache/`
Mail Template	`backend/app/integrations/processing/mail/`